Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ” Laborator Python: funcții, metode și importuri pe web”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

În acest laborator veți scrie programe Python care comunică cu un site web real și veți exersa trei concepte de bază: **importurile**, **metodele** și **funcțiile**. La final veți avea propriul modul Python, care generează un raport despre un site web.

După acest laborator veți putea:

- Să importați module din biblioteca standard și module externe și să explicați diferența dintre ele.
- Să apelați metode și să citiți atribute ale obiectelor create de alt cod (răspunsuri, șiruri de caractere, dicționare).
- Să scrieți propriile funcții, cu parametri, valori implicite, valori returnate, docstring-uri și tratarea erorilor.
- Să împărțiți codul într-un modul propriu și să îl importați dintr-un alt fișier.

Ce trebuie să știți înainte: variabile, instrucțiuni `if`, bucle `for`, liste și dicționare.

Site-ul țintă: toate exercițiile de citire folosesc `https://cybercor.org`. Exercițiile care *trimit* date folosesc `https://httpbin.org`, un serviciu public creat pentru testarea cererilor.

Importuri, metode și funcții, pe scurt

- Un **import** aduce în programul vostru cod scris de altcineva: import requests.
- O **metodă** este o funcție care aparține unui obiect și se apelează cu punct: response.json(), text.lower().
- O **funcție** este un bloc de cod cu nume, reutilizabil, pe care îl scrieți voi: def get_title(html): ....

Configurare și reguli

Înainte de exercițiul 1, instalați o bibliotecă externă și creați un director pentru laborator.

1. Verificați versiunea Python cu python --version (3.9 sau mai nouă).
2. Instalați biblioteca requests: pip install requests.
3. Creați un director numit web_lab și lucrați în el. Salvați fiecare parte într-un fișier separat: part1_imports.py, part2_methods.py și așa mai departe.
4. Începeți fiecare fișier cu acest șablon:

```
# Laborator: funcții, metode și importuri pe web
# Student: <numele vostru>


BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

Reguli de lucru

Lucrați cu un site web real, de care depind oameni reali. Respectați aceste reguli la fiecare exercițiu:

- **Folosiți mereu un timeout.** Fiecare cerere primește `timeout=TIMEOUT`, ca programul să nu se blocheze la nesfârșit.
- **Faceți pauze între cereri.** Când trimiteți mai multe cereri una după alta, puneți `time.sleep(1)` între ele.
- **Doar citire.** Către cybercor.org trimiteți doar cereri `GET` și `HEAD`. Tot ce trimite date (`POST`, parametri de interogare, antete personalizate pe care vreți să le verificați) merge către httpbin.org.
- **Nicio buclă fără limită.** Nu scrieți niciodată o buclă `while True:` care trimite cereri.
- **Rămâneți pe țintă.** Accesați doar URL-urile indicate în exerciții. Nu scanați alte site-uri.

DESFAȘURAREA LUCRARII

Partea 1 : Importuri (exerciții 1-8)

Din aceste exerciții am învățat cum să folosesc modulele și bibliotecile în Python pentru cereri web.

Am învățat diferența dintre instrumentele incluse nativ în Python (urllib) și cele create de comunitate (requests), precum și modurile de a le importa în cod (import, from ... import, alias-uri as

Am văzut cum pot inspecta conținutul unui modul cu dir(), cum citesc documentația direct din terminal cu help() și cum folosesc parametrul timeout pentru a preveni blocarea programului

Am învățat să cronometrezi durata cererilor web (time.perf_counter() vs response.elapsed) și să prind erorile de import (try / except ImportError) pentru a afișa mesaje utile când lipsește o bibliotecă

Rezultatul executări a exercițiilor :

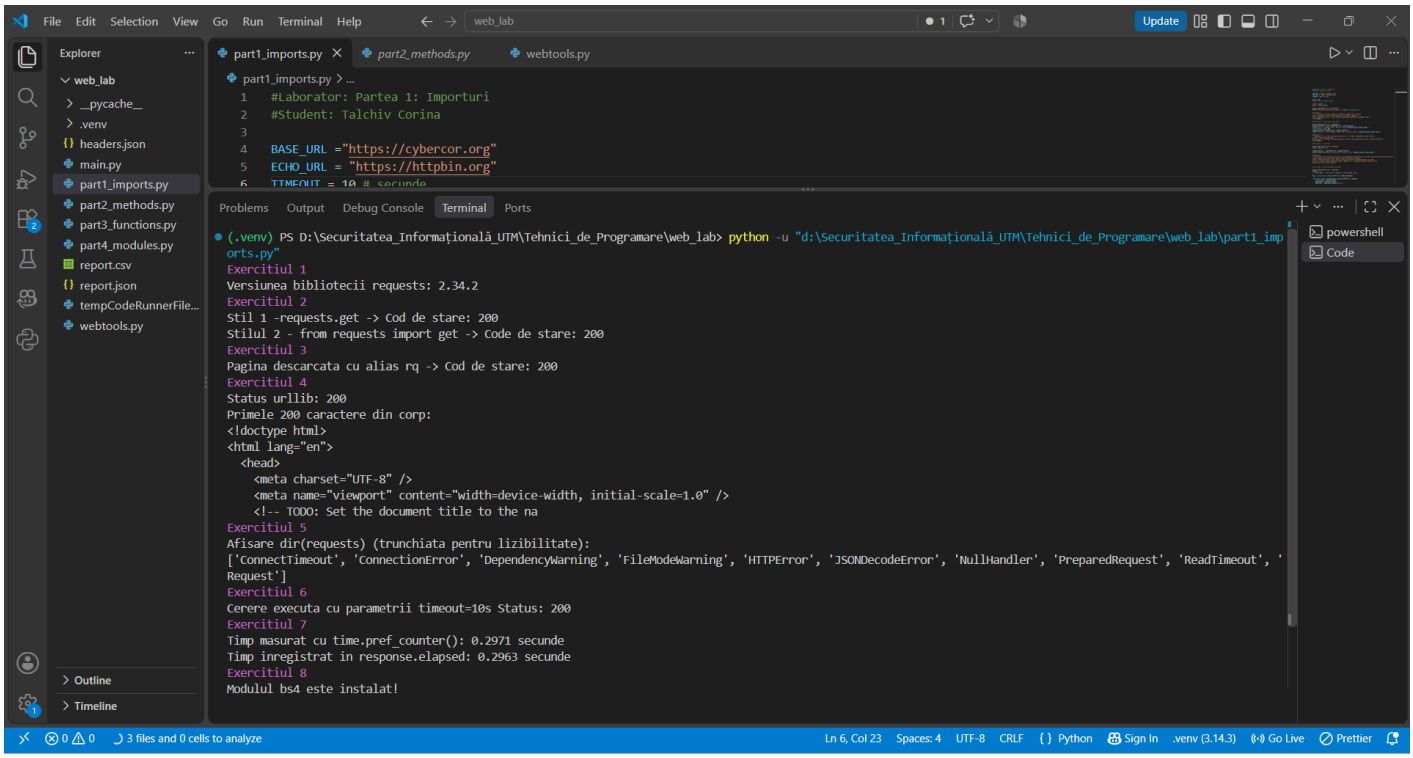


Figura 1 – Rezultate ale exercițiilor(1-8)

Partea 2 : Metode și atribute ale obiectelor web (exerciții 9-20)

Din această parte, am învățat cum să folosesc eficient instrumentele nativ integrate în obiectul Response din biblioteca requests, făcând o diferență clară între **atribute** (proprietăți/valori precum .status_code sau .url) și **metode** (acțiuni pe care le apelez cu paranteze, cum ar fi .json() sau .raise_for_status()).

- Am extras date despre pagină (status_code, ok, encoding) și am învățat să folosesc metoda .raise_for_status() într-un bloc try/except HTTPError pentru a trata elegant paginile inexistente (404), fără ca programul să se oprească cu eroare.
- Am lucrat cu Antetele și Dicționarele (Ex. 11–12): Am parcurs toate antetele HTTP transmise de server (response.headers) și am observat că dicționarul de antete din requests este de tip *case-insensitive* (nu ține cont de majuscule/minifuncții), deci "Server" și "server" returnează același rezultat.
- Am înlănțuit metode pe șiruri de caractere (ex: .text.lower().count("cyber")) deoarece fiecare metodă de string returnează un alt string peste care pot aplica o nouă metodă.
- Am extras titlul paginii dintre etichetele <title> și </title> folosind .find() și slicing ([start:end]).
- Am numărat liniile codului HTML cu .splitlines(), am aflat cea mai lungă linie cu max(lines, key=len) și am verificat securitatea legăturii HTTPS cu .startswith().
- Am urmărit lanțul de redirecționări automate de la http:// la https:// inspectând lista response.history.
- Am înțeles diferența dintre metoda HEAD (care descarcă doar antetele, având len(content) == 0) și GET (care descarcă întreg corpul paginii).
- Am inspectat cookie-urile primite (response.cookies) și am utilizat un obiect requests.Session() pentru a păstra antetele personalizate (User-Agent) pe parcursul mai multor cereri consecutive.

Rezultatul executări a exercițiilor :

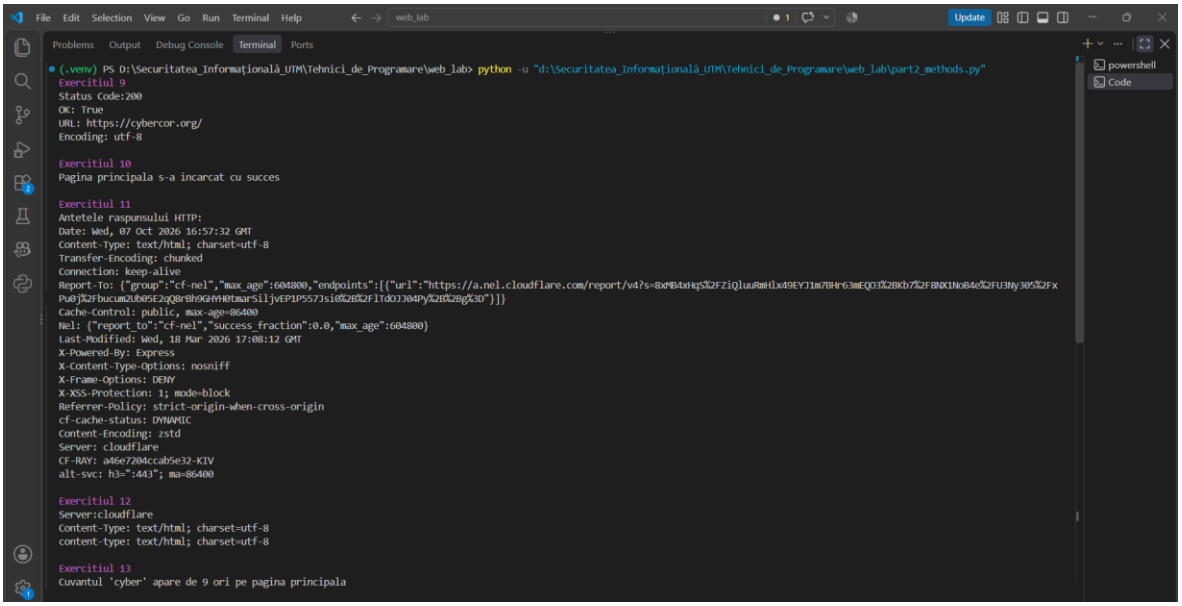


Figura 2 – Rezultatul executării 9-13

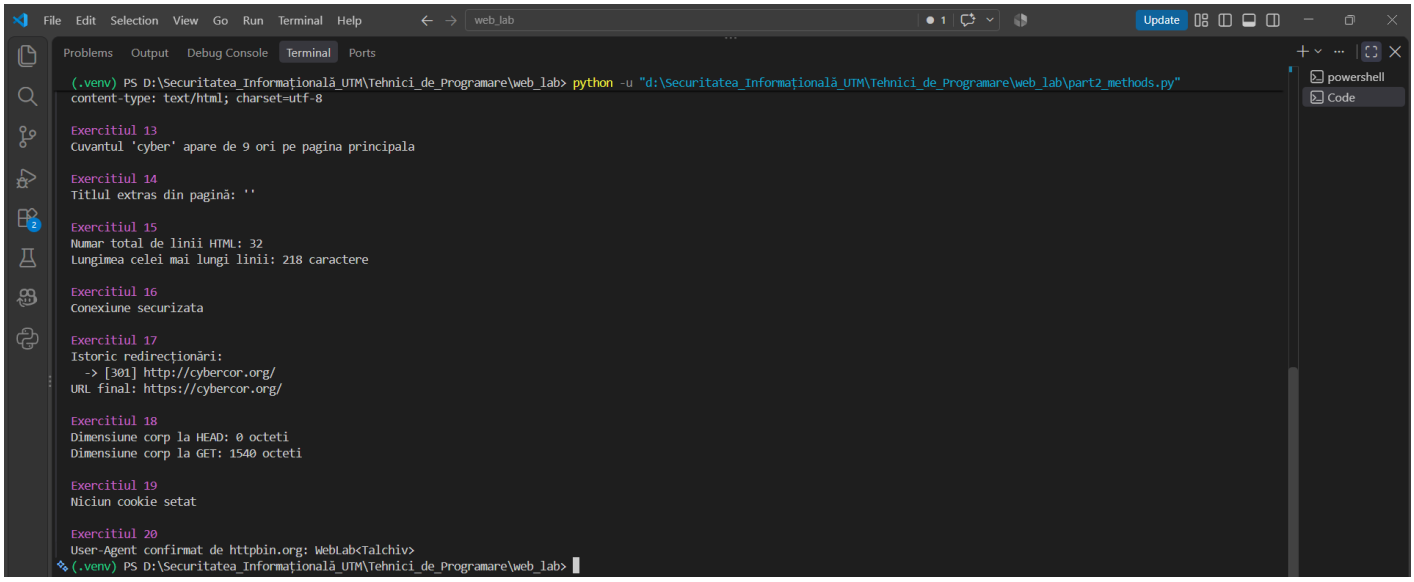


Figura 3 – Rezulatul executării a exercițiilor (13-20)

Partea 3 : Scrierea propriilor funcții (exerciții 21-34)

Am scris funcții precum fetch() și get_status(), adăugând parametri impliciți (timeout=10) și temporizări cu time.sleep(1) pentru a interoga mai multe rute fără a suprasolicita serverul

Am izolat extragerea titlului în get_title(html), o funcție care procesează doar text fără cereri HTTP.

Am adăugat documentație (docstring) și **Type Hints** (url: str -> int).

Am observat că Type Hints **nu opresc execuția programului** la rulare (Python nu le impune automat), ci sunt folosite de mediul de dezvoltare (IDE) și de linia de comandă pentru verificare.

Am creat page_exists() care prinde requests.RequestException și returnează False în caz de domenii invalide.

Am scris funcții care returnează dicționare (check_paths, security_headers) și am compus funcțiile între ele (score_headers(security_headers(BASE_URL))) pentru a evalua automat securitatea paginii (ex: scor "3/5")

Am creat fetch_robots() și disallowed_paths() gestionând în siguranță cazurile în care fișierul nu există (None).

Am folosit *urls (număr variabil de argumente poziționale) în response_times() și **details (argumente de tip *keyword*) în log(), pentru a formata mesaje dinamice de jurnalizare.

Rezultatul executări a exercițiilor :

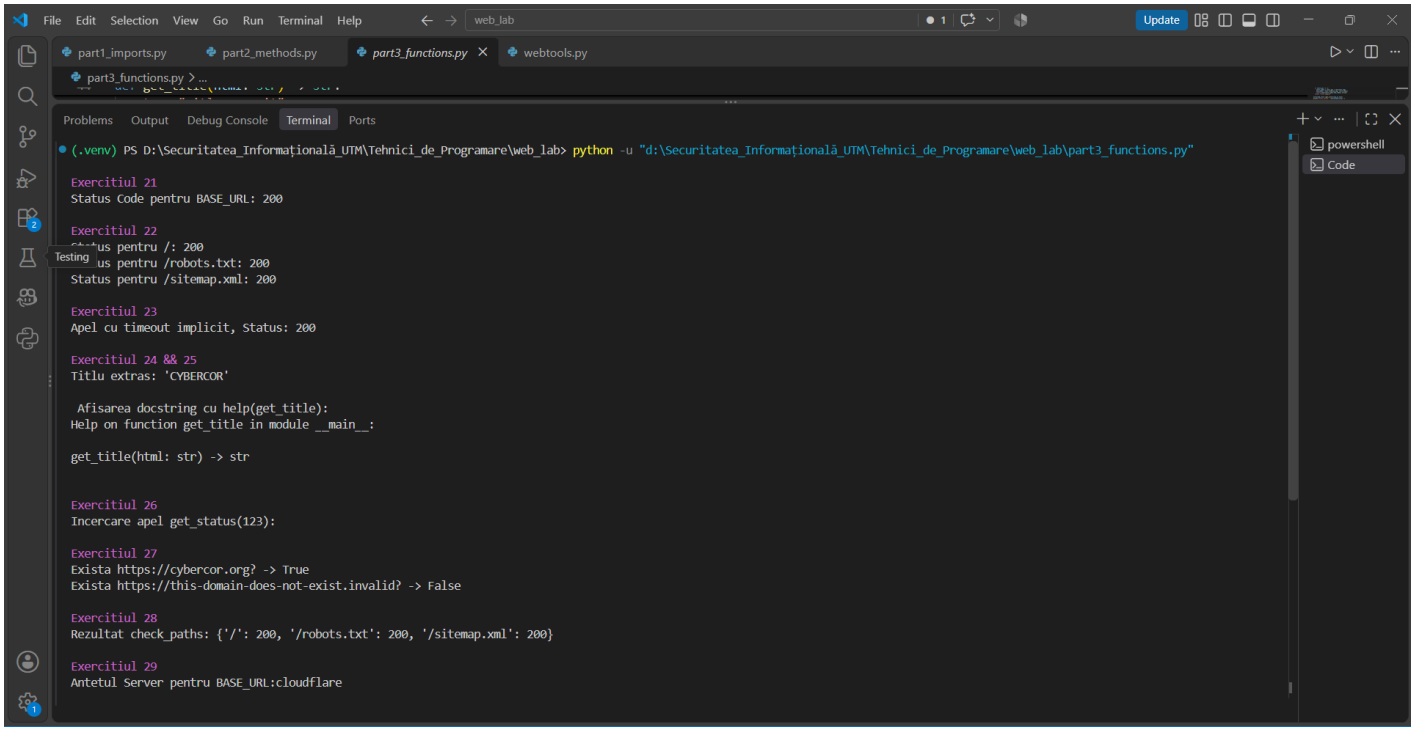


Figura 4 – Rezultatul exercițiilor (21-29)



Figura 5 – Rezultatul exercițiilor (30-34)

Partea 4 : Alte module din biblioteca sandard (exerciții 35-43)

În această parte am învățat cum să combin modulele standard din Python (urllib.parse, re, html.parser, hashlib, json, socket, ssl, datetime) cu biblioteca requests pentru a construi instrumente mai avansate de analiză web, securitate și inspecție de rețea

Am folosit urllib.parse.urlparse() pentru a descompune un URL în componentele sale (scheme, netloc, path, query, fragment).

Am folosit urljoin() pentru a converti automat link-urile relative (de ex. ../index.html) în legături absolute funcționale.

Am separat link-urile interne de cele externe comparând domeniul (netloc).

Am folosit expresii regulate (re.findall) pentru o extracție rapidă a link-urilor.

Am extins clasa html.parser.HTMLParser și am suprascris metoda callback handle_starttag() pentru a colecta sursele imaginilor (src). Am verificat corectitudinea parserului prin teste de tip assert.

Am generat un hash cryptographic SHA-256 (hashlib.sha256) peste conținutul binar al paginii (response.content). Rezultatele ar fi diferite dacă pagina conține conținut dinamic (timp, token-uri, reclame).

Am salvat antetele răspunsului într-un fișier headers.json utilizând json.dump() cu indent=2 și le-am citit înapoi cu json.load().

Am aflat adresa IP a domeniului folosind interogarea DNS din socket.gethostbyname().

Am creat o conexiune securizată SSL/TLS peste un socket direct și am extras data de expirare a certificatului HTTPS (notAfter), calculând numărul de zile rămase până la expirare.

Rezultatul executări a exercițiilor :

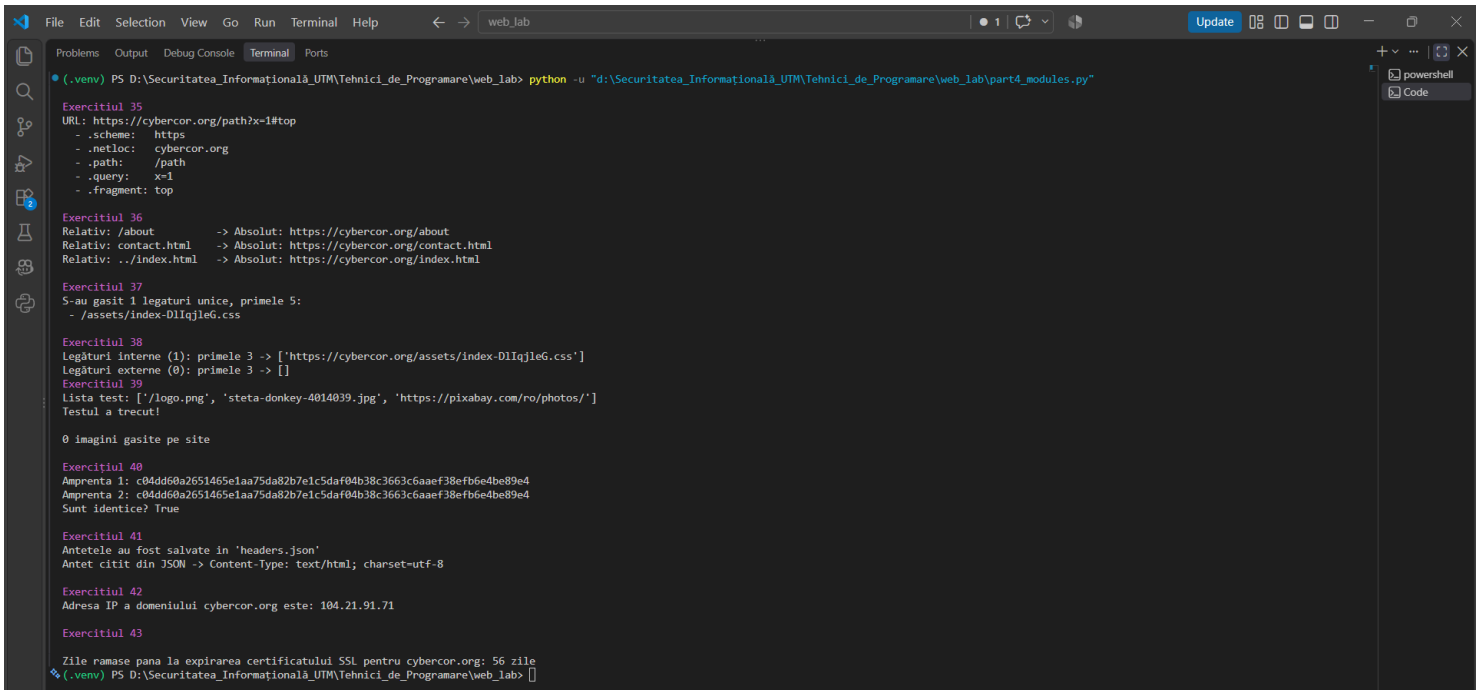


Figura 6 – Rezultatele executări a exercițiilor (35-43)

Partea 5 : Construirea propriului modul (exerciții 44-50)

În această parte am organizat aplicația într-o arhitectură modulară profesională, separând codul în două fișiere distincte: webtools.py (modulul restrâns cu unelte reutilizabile) și main.py (programul principal executabil)

Am mutat toate funcțiile de rețea, securitate și analiză HTML într-un fișier separat (webtools.py).

Am adăugat constanta DEFAULT_HEADERS = {"User-Agent": "WebLab-Talchiv_Corina"} pentru a efectua cereri HTTP identificate și sigure.

Am adăugat blocul if __name__ == "__main__": la final permițând rularea testelor automate doar atunci când fișierul este executat direct, nu și când este importat.

Am importat nume specifice folosind from webtools import ... și am adăugat un comentariu explicativ despre coliziunea numelor de funcții.

Am implementat modulul argparse pentru a permite rularea din linia de comandă cu parametri dinamici:python main.py https://cybercor.org

Am generat raportul report.csv cu modulul nativ csv, salvând verificările de căi împreună cu marcajul temporal ISO (datetime.now().isoformat()).

Am creat o funcție complexă care adună într-un singur apel: codul de stare, URL-ul final, titlul, adresa IP, lanțul de redirecționări, scorul antetelor de securitate, valabilitatea certificatului SSL, numărul de link-uri (interne/externe) și rutele interzise din robots.txt.

Datele sunt afișate în consolă sub formă de raport și exportate automat în report.json.

Rezultatul executări a exercițiilor :

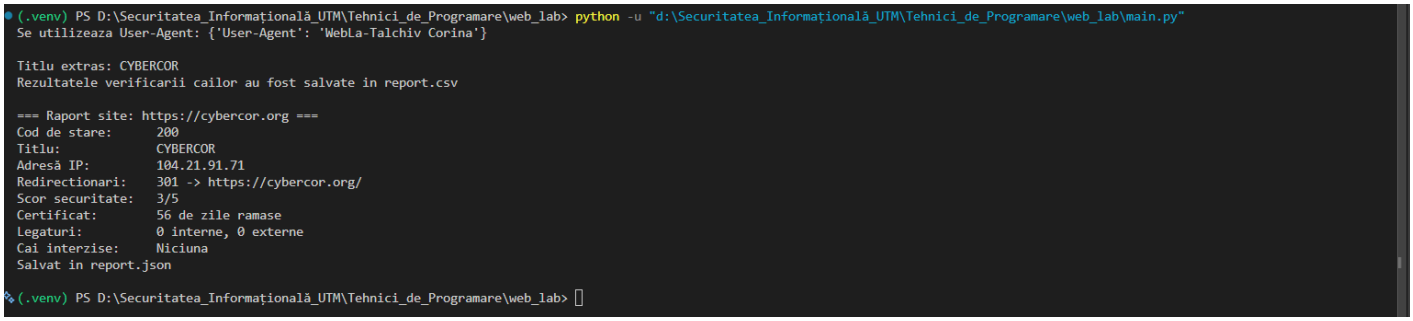


Figura 7 – Rezultatul din fișierul main

S-a generat report.json

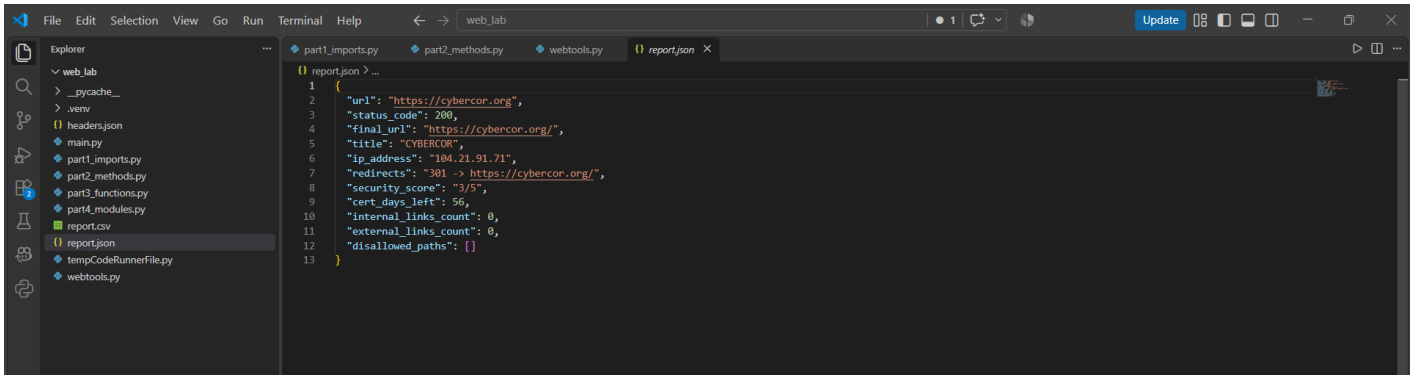


Figura 8 – Fișierul generat report.json

CONCLUZII

Prin parcurgerea acestui laborator, am făcut trecerea de la scrierea de scripturi simple la arhitecturarea unui modul Python complet, reutilizabil și orientat spre securitate web.

Cea mai importantă realizare a fost separarea clară a responsabilităților. Prin crearea modulului `webtools.py`, am încapsulat logica complexă de inspecție HTTP, analiză SSL, interogări DNS și procesare HTML, lăsând fișierul `main.py` să funcționeze ca un CLI (*Command Line Interface*) curat și eficient. Am integrat controlul din linia de comandă prin `argparse` și am automatizat generarea de rapoarte structurate în formate standardizate (`CSV` și `JSON`), transformând codul într-un instrument util pentru audituri rapide de securitate.